NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING

Midterm (2015-16)

CS3230: Design and Analysis of Algorithms

September 2015 Time Allowed: 90 Minutes

INSTRUCTIONS TO CANDIDATES

1. This midterm paper consists of **FOUR (4)** questions and **FOURTEEN (14)** printed pages including this page and two extra sheets of scratch paper. You may detach the scratch paper for use during the midterm.

2. Answer all questions. You have 90 minutes to earn 40 marks.

3. Write your answer in the space provided. If you need more space, write on the back of the sheet and indicate clearly that that the solution is being continued on the back of the sheet.

4. Do not spend too much time on any problem. Read them all through first and attack them in the order that allows you to make the most progress.

5. Show your work, as partial credit will be given. You will be graded on the correctness and efficiency of your answers, and also on your clarity. Be clear and always explain your reasoning.

6. This is a **CLOSED BOOK** midterm. You are allowed to bring in one handwritten (on both sides) A4 crib sheet and use an approved calculator.

For Examiner's Use Only		
	Max Marks	Earned Marks
Problem 1	16	
Problem 2	4	
Problem 3	12	
Problem 4	8	
TOTAL:	40	

Student number:

Problem 1. True or False, and Justify (16 Marks)

Circle **T** (true) or **F** (false) for each of the following statements and justify your answer. Your justification is worth more than your true or false answer. You will not receive any mark is the justification is poor or missing, even if you answer true or false correctly.

a) **T F**

$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(\ln n!)$ implies $T(n) = \Theta(n \log n)$.

b) **T F**

$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(4^{log_2 n})$ implies $T(n) = \Theta(n^3)$.

c) **T F**

$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^3)$ implies $T(n) = \Theta(n^3)$.

d) **T F**

$T(n) = T\left(\frac{n}{2}\right) + T(\sqrt{n}) + \Theta(n)$ implies $T(n) = O(n)$.

e) **T F**

Consider a variant of Fibonacci sequence defined as:

$G_0 = 0$, $G_1 = 1$ and $G_n = 2G_{n-1} + G_{n-2}$.

Computing G_n can be done in time $\Theta(\log n)$.

f) **T F**

It is possible to select the same input for both quicksort and randomized quicksort such that quicksort will have a runtime of $\Theta(n^2)$ but randomized quicksort will have an expected runtime of $\Theta(n \log n)$.

g) **T F**

A binary search tree can be constructed in linear time from a set of n random integers selected from $\{1,.., n^3\}$ in the worst case.

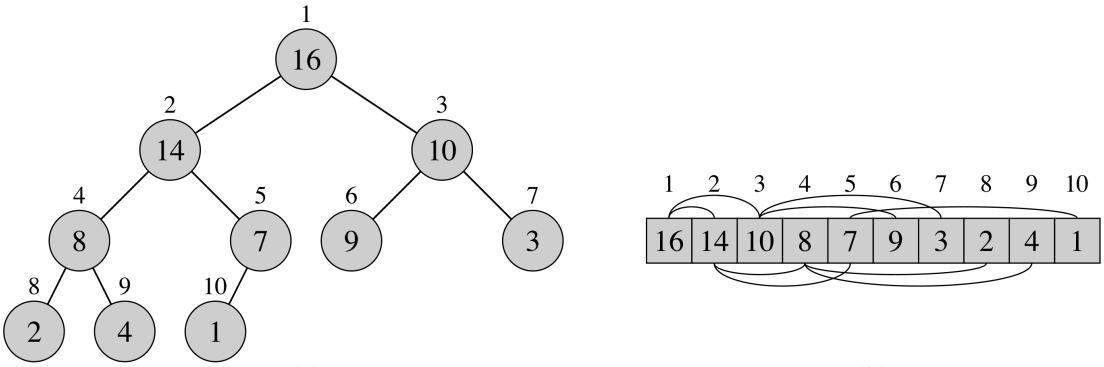
h) **T F**

Under a decision tree model, any algorithm for merging two sorted lists of length n each takes at least n comparisons in the worst case.

Problem 2. Search (4 Marks)

Given a sorted array $A[1,..,n]$ of distinct integers, give a $O(\log n)$ algorithm to find out whether there is an index i such that $A[i] = i$. You should justify why your algorithm returns the correct result and has runtime of $O(\log n)$. (Hint: the transformation $A[i] - i$ may be useful.)

Problem 3. Build Heap (12 Marks)



(a) (b)

A binary max-heap is a binary tree satisfying the property that the key of the parent is greater than or equal to the key of the children. It is often implemented with an array $A[1,..,n]$ where $A[1]$ is the root and the index of the left and right child of element i is $2i$ and $2i+1$ respectively. The tree and array representations of the heap are illustrated in parts (a) and (b) of the figure[1] above. For simplicity, you may assume that the heap is a *complete* binary tree with $2^{k+1} - 1$ elements for some height k, such that there are 2^k leaves at depth k.

Assume that you are given the procedure MAX-HEAPIFY(A,i) which takes in an array A and an index i in the array, and has the precondition that the subtrees rooted at the left and right subtrees of element i are max-heaps. After MAX-HEAPIFY(A,i) is run, the postcondition is that the subtree rooted at i is a max-heap. The runtime for MAX-HEAPIFY(A,i) is $\Theta(\log n)$ where n is the size of the subtree rooted at i. Consider the procedure:

BUILD-MAX-HEAP(A)

1. $n \leftarrow$ size(A)
2. **for** $i = \lfloor n/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A,i)

a) State the invariant for the loop in BUILD-MAX-HEAP

[1] From CLRS.

b) Using the invariant, prove the correctness of BUILD-MAX-HEAP.

We now consider a recursive version of BUILD-MAX-HEAP :

RECURSIVE-BUILD-MAX-HEAP(A,i)

```
n ← size(A)
if i > ⌊n/2⌋
    return
else
    RECURSIVE-BUILD-MAX-HEAP(A, 2i)
    RECURSIVE-BUILD-MAX-HEAP(A, 2i+1)
    MAX-HEAPIFY(A,i)
```

c) Argue that RECURSIVE-BUILD-MAX-HEAP correctly builds a max-heap.

d) Write the recurrence for RECURSIVE-BUILD-MAX-HEAP. What is its running time?

Problem 4. Universal Hashing (8 Marks)

A hash function maps from some universe U into m bins labeled with distinct labels from the set $L = \{1, 2, \ldots, m\}$. In universal hashing, we have a family of hash functions $H = \{h : U \rightarrow L\}$ with the following property:

For every $x, y \in U$, if $x \neq y$, then $Pr_{h \in H}\big(h(x) = h(y)\big) \leq 1/m$.

That is, any two keys in the set collide with probability no more than $1/m$ when the hash functions are drawn randomly from H.

Assume that a hash function is drawn randomly from H and has been used to hash n keys into the hash table that resolves collision by chaining.

a) A new key that is not currently in the table is drawn from U. What is the expected chain length of the bin that it will hash into?

b) What is the number of bins that should be used so that the expected number of collisions with a new key is $O(1)$?

c) An adversary is allowed to choose the n keys to be hashed into the hash table. What is the expected chain length of the bin that a new key hashes into, given that the adversary tries to maximize the expected length?

END OF EXAM

Scratch Paper

Scratch Paper